

DAY 18: PROBABILITY FUNDAMENTALS FOR DATA SCIENCE

60-Day Data Science Learning Journey | Muhammad Ahmad

Goal: Understand probability concepts used in Data Science, statistics and Machine Learning.

Prerequisites: Python, NumPy, Pandas and Statistics fundamentals from Days 1–17.

1. What is Probability?

Probability measures how likely an event is to happen. It ranges from 0 to 1, or 0% to 100%. 0 means impossible and 1 means certain.

2. Basic Probability Formula

For equally likely outcomes: $P(A) = \text{favorable outcomes} / \text{total possible outcomes}$. A fair die gives $P(4) = 1/6 \approx 0.1667$.

```
probability = 1 / 6
print(probability)
```

Expected / example output:

```
0.16666666666666666
```

Line-by-line:

Line 1 divides one favorable outcome by six possible outcomes.

Line 2 prints the probability.

3. Experiment, Outcome and Sample Space

An experiment is an action that produces a result. An outcome is one possible result. A sample space contains all possible outcomes.

```
sample_space = {1, 2, 3, 4, 5, 6}
print(sample_space)
```

Expected / example output:

```
{1, 2, 3, 4, 5, 6}
```

Line-by-line:

Line 1 stores every possible die result.

Line 2 displays the sample space.

4. Events

An event is one or more outcomes we care about. For a die, the event 'even number' is {2,4,6}, so its probability is $3/6 = 0.5$.

```
event_even = {2, 4, 6}
probability = len(event_even) / 6
print(probability)
```

Expected / example output:

```
0.5
```

Line-by-line:

Line 1 defines favorable outcomes.

Line 2 counts them and divides by six.

Line 3 prints the result.

5. Complement Probability

The complement means the event does not happen. $P(A^c) = 1 - P(A)$. If $P(\text{Pass}) = 0.7$, then $P(\text{Fail}) = 0.3$.

```
p_pass = 0.7
p_fail = 1 - p_pass
print(p_fail)
```

Expected / example output:

```
0.3
```

Line-by-line:

Line 1 stores P(Pass).

Line 2 subtracts it from 1.

Line 3 prints P(Fail).

6. Union and Intersection

Union means A OR B. Intersection means A AND B. Python sets use | for union and & for intersection.

```
A = {2, 4, 6}
B = {5, 6}
print("Union:", A | B)
print("Intersection:", A & B)
```

Expected / example output:

```
Union: {2, 4, 5, 6}
Intersection: {6}
```

Line-by-line:

Lines 1–2 define events.

Line 3 combines unique outcomes.

Line 4 keeps common outcomes.

7. Probability of a Union

For two events: $P(A \cup B) = P(A) + P(B) - P(A \cap B)$. The intersection is subtracted because overlap would otherwise be counted twice.

```
p_a = 0.5
p_b = 0.3
p_ab = 0.1
p_union = p_a + p_b - p_ab
print(p_union)
```

Expected / example output:

```
0.7
```

Line-by-line:

Lines 1–3 store the three probabilities.

Line 4 applies the formula.

Line 5 prints the result.

8. Conditional Probability

Conditional probability asks for the probability of A given B. $P(A|B) = P(A \cap B) / P(B)$. Example: 30 of 60 male students passed, so $P(\text{Pass}|\text{Male}) = 30/60 = 0.5$.

```
male_students = 60
male_passed = 30
probability = male_passed / male_students
print(probability)
```

Expected / example output:

```
0.5
```

Line-by-line:

Line 1 stores the conditioning group size.

Line 2 stores favorable cases within that group.

Line 3 divides favorable cases by the conditioning group.

Line 4 prints the probability.

9. Why Conditional Probability Matters in Data Science

Predictive questions often have the form $P(Y|X)$, meaning the probability of a target Y given observed features X . For example, $P(\text{Pass} | \text{Study Hours, Attendance})$ expresses a prediction conditioned on student features.

10. Independent Events

Events are independent when one event does not change the probability of the other. For independent events, $P(A \cap B) = P(A)P(B)$. Two separate coin tosses are a common example.

```
p_heads = 0.5
p_two_heads = p_heads * p_heads
print(p_two_heads)
```

Expected / example output:

```
0.25
```

Line-by-line:

Line 1 stores the probability of Heads.

Line 2 multiplies independent probabilities.

Line 3 prints 25%.

11. Dependent Events

Events are dependent when one event changes the probability of another. Drawing two balls without replacement is a classic example because the first draw changes the remaining contents.

12. Random Variables

A random variable maps random outcomes to numerical values. Example: X = number of Heads in two coin tosses. X can be 0, 1 or 2.

13. Discrete vs Continuous Random Variables

Discrete variables have countable values, such as number of students, defects or Heads. Continuous variables can take values throughout a range, such as height, weight, temperature, time and distance.

14. Probability Distributions

A probability distribution assigns probabilities to possible values of a random variable. For two fair tosses: $X=0 \rightarrow 0.25$, $X=1 \rightarrow 0.50$, $X=2 \rightarrow 0.25$. The probabilities must sum to 1.

```
values = [0, 1, 2]
probabilities = [0.25, 0.50, 0.25]
print(sum(probabilities))
```

Expected / example output:

```
1.0
```

Line-by-line:

Lines 1–2 store values and probabilities.

Line 3 checks that total probability is 1.

15. Expected Value

Expected value is the probability-weighted long-run average: $E(X) = \sum xP(x)$. For two fair tosses, $E(X) = 0(0.25) + 1(0.50) + 2(0.25) = 1$.

```

values = [0, 1, 2]
probabilities = [0.25, 0.50, 0.25]
expected_value = sum(x * p for x, p in zip(values, probabilities))
print(expected_value)

```

Expected / example output:

```
1.0
```

Line-by-line:

Lines 1–2 store values and probabilities.

Line 3 pairs each value with its probability, multiplies them and sums the products.

Line 4 prints the expected value.

16. Probability Simulation with NumPy

Simulation estimates probability by repeating a random experiment many times. The exact result changes each run.

```

import numpy as np

results = np.random.choice(["Heads", "Tails"], size=1000)
heads = np.sum(results == "Heads")
tails = np.sum(results == "Tails")

print("Heads:", heads)
print("Tails:", tails)

p_heads = heads / 1000
print("Estimated probability:", p_heads)

```

Expected / example output:

```

Example:
Heads: 503
Tails: 497
Estimated probability: 0.503

```

Your output will vary.

Line-by-line:

Line 1 imports NumPy.

Line 3 simulates 1,000 tosses.

Lines 4–5 count each result.

Lines 7–8 print counts.

Line 10 estimates P(Heads).

Line 11 prints the estimate.

17. Probability from a Pandas DataFrame

A Boolean Series contains True/False values. True behaves like 1 and False like 0 in arithmetic, so the mean of a Boolean column gives the proportion that are True.

```

import pandas as pd

data = {
    "Name": ["Ali", "Usman", "Hamza", "Ahmad", "Bilal"],
    "Marks": [85, 72, 91, 45, 65],
    "Passed": [True, True, True, False, True]
}

df = pd.DataFrame(data)
p_pass = df["Passed"].mean()
print(p_pass)

```

Expected / example output:

```
0.8
```

Line-by-line:

Lines 1–2 import Pandas.

Lines 4–8 create the dataset.

Line 10 creates the DataFrame.

Line 11 calculates the passing proportion.

Line 12 prints 0.8, or 80%.

18. Conditional Probability with Student Data

To calculate $P(\text{Pass}|\text{CS})$, first filter to CS students, then calculate the passing proportion inside that subset.

```
data = {
    "Name": ["Ali", "Usman", "Hamza", "Ahmad", "Bilal"],
    "Department": ["CS", "CS", "SE", "CS", "SE"],
    "Marks": [85, 72, 91, 45, 65],
    "Passed": [True, True, True, False, True]
}

df = pd.DataFrame(data)
cs_students = df[df["Department"] == "CS"]
p_pass_given_cs = cs_students["Passed"].mean()
print(p_pass_given_cs)
```

Expected / example output:

```
0.6666666666666666
```

Line-by-line:

CS students are Ali, Usman and Ahmad.

Two of the three passed, so $P(\text{Pass}|\text{CS})=2/3=0.667$.

19. Probability vs Statistics

Probability often starts with a model or assumptions and asks what outcomes are likely. Statistics often starts with observed data and asks what the data reveal about an underlying population. They are closely connected in Data Science.

DAY 18 PRACTICE TASKS

1. Roll a fair die. Calculate $P(1)$.
2. Calculate $P(\text{even number})$ on a fair die.
3. Toss a coin twice. Calculate $P(\text{Heads twice})$.
4. If $P(\text{Pass})=0.85$, calculate $P(\text{Fail})$.
5. If $P(A)=0.6$, $P(B)=0.4$ and $P(A \cap B)=0.2$, calculate $P(A \cup B)$.
6. If 80 CS students exist and 60 passed, calculate $P(\text{Pass}|\text{CS})$.
7. Simulate 1,000 coin tosses with NumPy and estimate $P(\text{Heads})$.
8. Create a student DataFrame and calculate the probability of passing.
9. For marks [85,72,91,45,65], calculate $P(\text{Marks} \geq 80)$.
10. Build a Student Probability Analyzer for total, passed, failed, $P(\text{pass})$, $P(\text{fail})$, $P(\text{marks} \geq 80)$ and $P(\text{marks} \geq 70)$.

PRACTICE TASK SOLUTIONS

1. $1/6 \approx 0.1667$
2. $3/6 = 0.5$
3. $0.5 \times 0.5 = 0.25$
4. $1 - 0.85 = 0.15$
5. $0.6 + 0.4 - 0.2 = 0.8$
6. $60/80 = 0.75$
7. Count simulated Heads and divide by 1,000. Exact output varies.
8. Four of five students passed, so $P(\text{pass})=0.8$.
9. Two of five marks are at least 80, so $P=0.4$.
10. Use the complete project below.

FULL MINI PROJECT: STUDENT PROBABILITY ANALYZER

```
import pandas as pd

data = {
    "Name": ["Ali", "Usman", "Hamza", "Ahmad", "Bilal", "Hassan", "Zain", "Omar"],
    "Department": ["CS", "CS", "SE", "SE", "CS", "SE", "CS", "SE"],
    "Marks": [85, 72, 91, 45, 65, 88, 95, 70]
}

df = pd.DataFrame(data)

df["Passed"] = df["Marks"] >= 50

total_students = len(df)
passed_students = df["Passed"].sum()
failed_students = (~df["Passed"]).sum()

p_pass = passed_students / total_students
p_fail = failed_students / total_students
p_80 = (df["Marks"] >= 80).mean()
p_70 = (df["Marks"] >= 70).mean()

print("Total Students:", total_students)
print("Passed Students:", passed_students)
print("Failed Students:", failed_students)
print("Probability of Passing:", p_pass)
print("Probability of Failing:", p_fail)
print("P(Marks >= 80):", p_80)
print("P(Marks >= 70):", p_70)
print(df)
```

Expected results:

```
Total Students: 8
Passed Students: 7
Failed Students: 1
Probability of Passing: 0.875
Probability of Failing: 0.125
P(Marks >= 80): 0.375
P(Marks >= 70): 0.75
```

Line-by-line project explanation

1. Import Pandas.
2. Create student names, departments and marks.
3. Convert the dictionary into a DataFrame.
4. Create Passed by checking Marks >= 50.
5. len(df) counts students.
6. sum() counts True values, giving passed students.
7. ~ reverses the Boolean values so sum() counts failed students.
8. Divide passed students by total students to estimate P(Pass).
9. Divide failed students by total students to estimate P(Fail).
10. The mean of Marks >= 80 gives the proportion meeting the 80 threshold.
11. The mean of Marks >= 70 gives the proportion meeting the 70 threshold.
12. Print the probability results and dataset.

COMMON MISTAKES

- Confusing an outcome with an event.
- Forgetting probabilities must be between 0 and 1.
- Using $P(A)+P(B)$ for a union when events overlap.
- Confusing $P(A|B)$ with $P(B|A)$.
- Assuming events are independent without justification.
- Treating a simulation estimate as exactly equal to theoretical probability.
- Assuming an unusual observation is automatically an error.

QUICK REVISION

```
Probability → likelihood of an event
Experiment → action producing an outcome
Outcome → one possible result
Sample Space → all possible outcomes
Event → one or more outcomes of interest
Complement → event does not happen
Union → A OR B
Intersection → A AND B
Conditional Probability →  $P(A|B)$ 
Independent Events → one event does not change the other
Random Variable → numerical representation of a random outcome
Probability Distribution → probabilities assigned to possible values
Expected Value → probability-weighted long-run average
```

FINAL CHECKLIST

- I can define probability.
- I can calculate basic probability.
- I understand experiment, outcome, sample space and event.

- I can calculate complements.
- I understand union and intersection.
- I can use the union formula.
- I understand conditional probability.
- I can distinguish independent and dependent events.
- I understand random variables.
- I can distinguish discrete and continuous variables.
- I understand probability distributions.
- I can calculate expected value.
- I can simulate probability with NumPy.
- I can calculate simple probabilities from Pandas data.
- I completed the Student Probability Analyzer.